

# Core Theory & Terminology

## 1. The Bottleneck of the Naive DFT

The standard DFT is an  $O(N^2)$  algorithm. For every output element, you have to look at every input element. On GPUs, this is terrible because it wastes massive amounts of compute.

## 2. Cooley-Tukey & The "Butterfly"

**The Cooley-Tukey** algorithm reduces complexity to  $O(N \log N)$ . It does this by recursively splitting the DFT into smaller and smaller DFTs (e.g., splitting a length- $N$  transform into two length- $N/2$  transforms).

- **Bit-Reversal:** Since the array is recursively divided into even and odd indices, the data becomes physically scrambled. To facilitate efficient in-place computation of the FFT, it is necessary to initially load the data utilising "bit-reversed" indices.
- **The Butterfly:** The fundamental computational step involves combining two smaller sub-problems. This process includes a complex multiplication, performed by a Twiddle Factor, followed by an addition or subtraction operation.

## 3. Twiddle Factors ( $W_N^k$ )

When you divide a large FFT into smaller sections, simply combining the results won't work. Shifting the data in time affects its phase in the frequency domain, so you need to adjust the phase accordingly. **Twiddle factors** are complex phase rotations, made of sines and cosines, used during butterfly computations.

## 4. Bailey's 6-Step Matrix Factorisation

As  $N$  gets huge, you can no longer fit the array into the GPU's ultra-fast SRAM (Shared Memory/Registers). If you try to use the standard Cooley-Tukey in Global Memory (VRAM), the memory access patterns become huge, strided jumps that crush memory bandwidth.

- **Bailey's Solution:** Treat a 1D array of length  $N$  as a 2D matrix of size  $N_1 \times N_2$ .
- **The 6 Steps:** 1. Transpose the matrix. 2. Do small FFTs on the rows. 3. Multiply by Bailey Twiddle factors. 4. Transpose again. 5. Do small FFTs on the rows again. 6. Transpose back.
- **Why it works:** By transposing the data, the sub-FFTs always read **contiguous** memory blocks, keeping the GPU's memory bandwidth fully saturated.

## 5. Tensor Cores & tcFFT (Radix-16)

Standard Cooley-Tukey (Radix-2) uses scalar math (1 multiply, 1 add). Modern GPUs have **Tensor Cores**, massive hardware units dedicated solely to matrix multiplication (Matmul). To use them, we switch to **Radix-16 tcFFT**, converting the FFT into a sequence of small  $16 \times 16$  matrix multiplications to max out the GPU's FLOPs.

## Kernel Breakdown

### F1: The Baseline (Dense Matmul)

- **What it is:** The mathematical definition of a DFT represented as a single dense Matrix Multiplication:  $Y = X \times W^T$ .
- **Implementation:** Complex numbers mean one multiplication becomes four real multiplications:  $(A + Bi)(C + Di) = (AC - BD) + (AD + BC)i$ . We use four `tl.dot()` calls to compute this.
- **Why we do it:** It verifies our complex matmul helper (`_cdot`) works, but it's  $O(N^2)$ , so it scales horribly.

### F2: Radix-2 Cooley-Tukey

- **What it is:** The classic  $O(N \log N)$  FFT algorithm running entirely in ultra-fast GPU registers.
- **Implementation Mechanics:**
  1. **Bit-reversal load:** It reads from global memory using the pre-computed `perm_ptr` to load data in bit-reversed order.
  2. **In-Register Loop:** For  $\log_2(N)$  stages, it does the butterfly. It uses `tl.gather` with an XOR mask (`partner = offs ^ half_step`) to find the partner register.
  3. **No SRAM:** Triton handles the register allocation. Because there is no shared memory sync (`tl.debug_barrier`), it is incredibly fast but structurally limited by the maximum number of registers a GPU thread block can hold (it starts failing past  $N \sim 16,384$ ).

### F3: Bailey 6-Step Pipeline

- The function involves subdividing a larger FFT into two smaller F2 FFTs to circumvent the register limitations inherent to F2.
- Mechanisms of implementation include utilising Kernel Fusion, which reduces the Number of global memory accesses from six to four steps, thereby optimising performance.
  - Specifically, the F2-A Fusion entails integrating the `bailey_scale_kernel` (Step 3) directly into the conclusion of the initial F2 execution through the application of the `BAILEY_EPILOGUE=True` flag.

- Additionally, the F2-B Fusion omits the final Transpose (Step 6) by directing the secondary F2 run to store its output in a transposed format via `STRIDED_STORE=True`.

## F4: tcFFT Radix-16

- **What it is:** Shifts the math from scalar operations (ALUs) to Tensor Cores using `tl.dot()`. Used specifically for  $N = 256$ .
- **Implementation Mechanics:**
  - Views length-256 as a  $16 \times 16$  grid ( $16^2 = 256$ ).
  - **Stage 0:** Reshapes to 3D, moves the axis to the end, flattens to 2D, and uses `tl.dot()` with the `_Fourier` matrix to transform the first base-16 digit.
  - **Stage 1:** Swaps the axes, broadcasts and multiplies the Radix-16 twiddle factors, flattens, and uses `tl.dot()` to transform the second base-16 digit.
  - This kernel is very sensitive to shape dimensions because `tl.dot()` mathematically requires strict  $M \times K$  and  $K \times N$  2D dimensions.

## F5: Bailey + F4

- **What it is:** Bridging the gap. It is identical in structure to F3, but it applies Bailey's 6-step factorisation for  $N = 65,536$  by using the tensor-core-accelerated **F4** as the sub-FFT instead of F2.
- **Implementation:** It simply calls `_transpose`, `_fft_chunk` (which maps to F4), `_scale`, etc., in sequence. No fancy fusion here, just proving the logic works at scale.

## F6: Recursive Bailey

- **What it is:** A recursive Python loop that can handle *any* arbitrarily massive size of  $N$  by chunking it down.
- **Implementation Mechanics:** It uses `f6_factor` to greedily break  $N$  into. It processes chunks of 256 (for F4), then 16 (for padded F1), then the leftovers. It builds the 6-step pipeline recursively, calling itself until it reaches a leaf node (a base chunk), at which point it runs the base FFT.
- **Problem:** It results in an enormous number of intermediate `_transpose` and `_scale` kernel launches, wasting Global Memory bandwidth.

## F7: Fused Recursive Bailey

- **What it is:** The final boss. F7 implements the same recursive math as F6 but heavily optimises memory bandwidth via **Kernel Fusion**.
- **Implementation Mechanics:** \* Every non-leaf level of Bailey has a `Scale` followed by a `Transpose (T2)`. F7 fuses this: it tells the Scale kernel to write its output to memory in a transposed pattern (`STORE_T=True`), entirely eliminating the T2 kernel launch.

- It does the same for the inner **FFT** and **Transpose (T3)**, saving yet another massive trip to VRAM.
- Because the floating-point math follows the same sequence as F6 (just stored to memory differently), F7 guarantees **bitwise equality** with F6 while running much faster.